

自动化与智能科学概论

Introduction to Automation and Intelligence Science

刘景泰 主编

2024 年 6 月

目录（CONTENTS）

自动化与智能科学概论	1
目录（CONTENTS）	2
第 6 章 强化学习	3
6.1 强化学习针对的问题	3
6.1.1 强化学习应用实例	3
1. 多级倒立摆系统	4
2. 视频游戏	4
3. 围棋大师	5
4. 双足机器人学习走路	5
5. 核聚变控制 ^[6]	6
6. ChatGPT ^[7]	7
7. 星际争霸大师 ^[8]	8
6.1.2 强化学习能解决的问题	9
6.2 强化学习与其他方法的区别和联系	10
6.2.1 强化学习与监督学习的区别	10
6.2.2 强化学习与监督学习的联系	11
6.2.3 强化学习与其他优化方法的区别	12
6.3 强化学习的思想精髓	13
6.4 强化学习的发展方向	14
6.4.1 强化学习算法的发展回顾	14
6.4.2 现有强化学习算法的分类	15
6.4.3 强化学习的发展趋势	16
1. 强化学习算法与深度学习的结合更加紧密	16
2. 强化学习算法与专业知识融合更加紧密	16
3. 强化学习算法与脑科学和认知神经科学的联系更加紧密	16
4. 强化学习算法理论分析会更强，算法会更稳定和高效。	16
6.5 常用的强化学习算法简介	17
6.5.1 Q-learning 算法	17
6.5.2 DQN 算法	18
6.5.3 策略梯度算法	19
6.5.4 深度确定性策略梯度算法	20
6.5.5 近端策略优化算法	21
6.6 强化学习的术语与总结	23
6.6.1 相关的术语	23
6.6.2 总结	24
6.7 思考	25
6.8 习题	25
参考文献	25

第6章 强化学习

在前述章节中已经介绍了常见的机器学习方法有监督学习、无监督学习、半监督学习、强化学习等。换言之，强化学习与监督学习、非监督学习并列构成机器学习的三大核心。

强化学习强调的是与环境的交互，通过持续地自主探索以找到更优的动作，进而取得最大化的预期利益。强化学习与机器学习其他领域的关系：

- 强化学习与监督学习和非监督学习同为机器学习的重要子领域。
- 监督学习寻找输入到输出之间的映射，比如分类和回归问题。
- 非监督学习旨在发现数据间的潜在关系，如进行聚类 and 降维。
- 强化学习通过与环境交互自主探索最优决策，比如动作控制和棋类决策

强化学习的主要特点，如图 6-1 所示：

- 智能体（agent）与环境（environment）的交互：智能体需要在环境中执行动作并获得反馈。
- 奖励回馈（reward）的延迟性：需要完成一系列动作（action）后，才能获知整体的效果，因而奖励反馈具有延迟性。
- 通过不断的试错探索，吸取经验教训，不断优化策略以获得更佳的奖励。

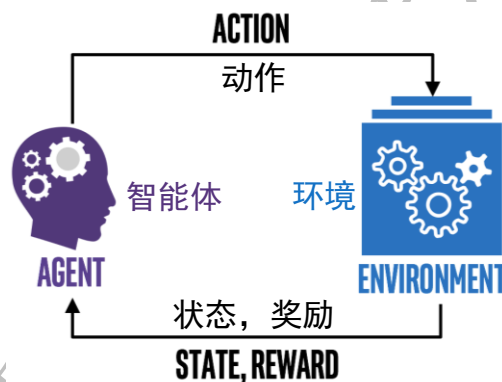


图 6-1 强化学习的特点

强化学习使得机器能够自主学习和做出决策，完成复杂任务，并能解决一些监督学习和非监督学习难以应对的问题。强化学习已经广泛应用于游戏设计、推荐系统、金融股票分析、交通管理等多个领域，特别是在机器人控制领域潜力巨大。本章将深入探讨强化学习所面对的问题、与其它学习方法的区别、核心理念、发展趋势、以及常用算法等多个方面。

6.1 强化学习针对的问题

本节首先通过应用实例分析强化学习算法在典型任务中的作用，然后总结这些应用实例的共同特点，进而找出强化学习算法比较擅长解决的问题类型。

6.1.1 强化学习应用实例

本节给出 7 个强化学习应用实例，包括多级倒立摆系统、视频游戏、围棋大师、机器人学习走路、核聚变控制、ChatGPT 和星际争霸大师。

1. 多级倒立摆系统

多级倒立摆的控制问题是典型的非线性控制问题，如图 6-2 为非线性二级倒立摆系统^[1]，该系统包括台车（图中黑色的矩形块）和二级摆（图中红色的串联杆），其中台车是可以控制的，二级摆是被动的。该系统的任务是控制台车左右运动，使得二级摆的末端稳定到竖直向上的位置且末端点与图中‘+’号重合。

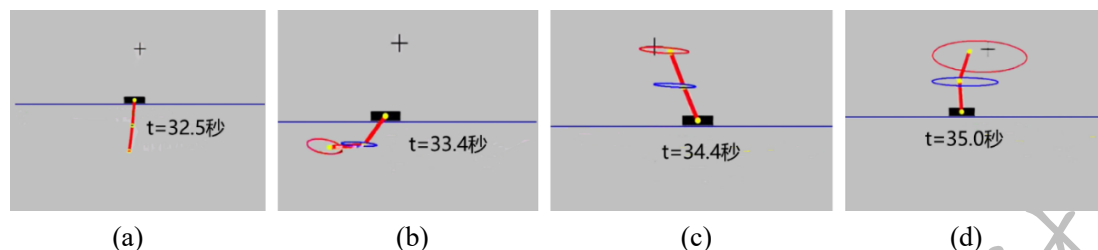


图 6-2 非线性二级倒立摆系统

针对二级倒立摆的控制问题，传统的控制器设计一般是基于模型的：首先对二级倒立摆系统利用拉格朗日方程建立动力学模型，然后基于该动力学方程设计非线性控制器以达成控制目标，目前基于模型的非线性控制器已经可以实现三级倒立摆的控制。

二级倒立摆的控制也可以使用强化学习算法来实现。与传统控制器设计方法不同，强化学习方法不需要推导系统的动力学方程，也不需要根据模型来设计控制器，对于一些建模困难的复杂系统而言非常关键。

针对二级倒立摆的控制，强化学习算法需要构建一个环境模拟器和策略网络，策略网络产生动作来驱动环境变化（如图 6-2（a）-（d）所示）；环境模拟器将状态、回报等信息反馈给策略网络以评判其动作的好坏并指导其向更好的方向更新；重复进行该双向传输过程，策略网络会逐渐收敛到最优状态。

需要注意的是，在该问题中所谓的策略是指状态空间到动作空间的映射，即在任何一个状态处，策略网络都能给出一个具体的动作。另外，所谓最优是指智能体经过一系列的序贯动作后可以达到目标。换言之，最终目标的实现是一些列动作的结果，根据贝尔曼最优性原理，这就要求智能体在每一个状态处的动作都是最优的。

2. 视频游戏

如图 6-3 为雅达利打砖块的视频游戏，是雅达利系列经典游戏中的一款。在该游戏中，玩家可以操控窗口底部的球拍左右移动，击打红色的小球，小球与窗口上部的彩色砖块层碰撞，将砖块打掉，进而得分。

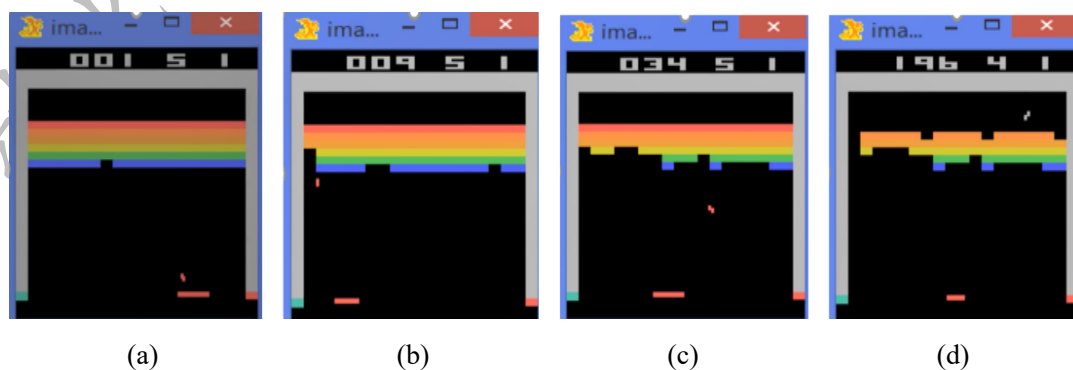


图 6-3 雅达利游戏打砖块游戏

2013 年 DeepMind 公司将强化学习与深度学习技术结合提出 DQN（Deep Q-Network）算法^[2]，并利用 DQN 训练深度神经网络控制球拍的左右移动，经过 4 个小时的训练得到了超越专业玩家水平的策略。DQN

算法不只在该游戏上取得超越专业玩家的水平, 移植到雅达利其他多款经典游戏中也取得超越人类玩家平均水平的成绩。正是由于深度强化学习算法在视频游戏上取得如此惊人的结果, DeepMind 于 2014 年被谷歌收购。同时, DQN 推动人工智能进入到深度强化学习时代。

具体而言, DQN 算法使用卷积神经网络构建了一个深度行为值函数网络, 该网络的输入是连续的 4 帧图像, 输出为该状态下 16 个动作对应的行为值函数。在学习过程中, 智能体通过由值函数构建的策略与雅达利模拟器进行交互, 产生一些列数据。智能体根据这些数据, 基于 Qlearning 算法优化值函数网络。如图 6-3 所示, 智能体学会了在每个状态处控制球拍左右运动的策略。

3. 围棋大师

围棋起源于中国, 博弈双方分别执白子和黑子, 在 19×19 的棋盘上轮流落子, 按照围棋的规则判断胜负。

围棋盘上有 361 个交叉点, 每个点有三种可能状态 (黑、白、无子), 理论上的状态空间可以达到 3^{361} 种配置。虽然并不是所有的配置都是合规的围棋局面 (因为有些违反了围棋的规则), 但即便是合规的局面数也非常巨大, 甚至超过宇宙中估计的原子总数 (大约为 10^{80})。因此, 这种庞大的状态空间使得围棋成为一个极具挑战的游戏, 对于传统棋类的搜索算法和人类的直觉来说都非常复杂。这也是为什么围棋在很长一段时间内被认为是人工智能研究中的一个重大挑战, 直到 2016 年 DeepMind 公司开发的 AlphaGo^[3] 诞生。

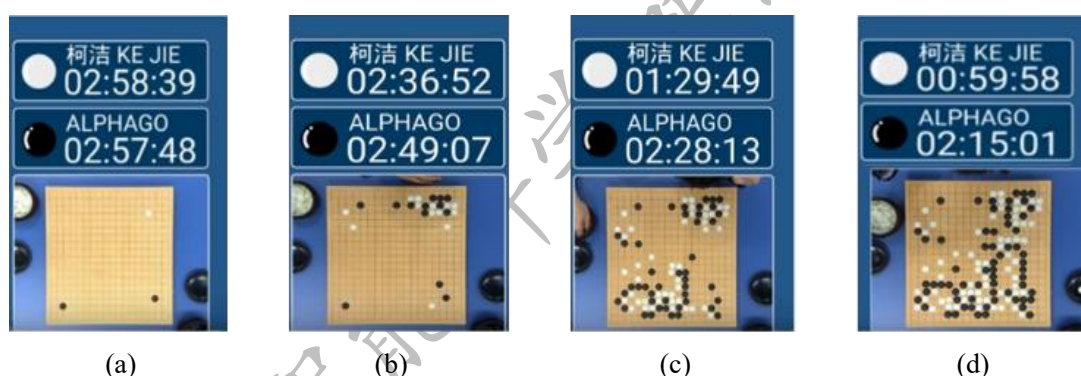


图 6-4 2017 年 AlphaGo Master 围棋机器人与围棋世界冠军柯洁对弈的过程

AlphaGo 设置了策略网络和价值网络, 使用自对抗强化学习算法优化策略网络和价值网络。具体而言, 训练过程分为自对抗数据产生过程和策略优化过程。

在自对抗过程中的每一步, 智能体使用蒙特卡洛树搜索技术通过前向前向仿真和模拟得到当前状态下最优的策略, 在蒙特卡洛树搜索的过程中, 通过策略网络剪枝树的宽度, 通过值网络剪枝树的深度; 然后利用蒙特卡洛树策略采样动作作用于围棋对弈; 每一步重复这个步骤, 同时将蒙特卡洛树策略及每一步的状态信息保存下来, 直到本局对抗结束, 得到胜负数据, 将这些数据保存起来。

在策略优化过程中, 利用自对抗数据进行策略网络和价值网络的优化。其中策略网络的优化利用的是保存的蒙特卡洛策略数据, 值网络的优化利用的是每局胜负数据。

2016 年之后, AlphaGo 技术形成了一个系列, 如未使用人类专家数据的 AlphaGo Zero^[4], 可以解决象棋、围棋、日本将棋的 AlphaZero, 以及具有学习效率高于模型强化学习的 MuZero^[5]。

4. 双足机器人学习走路

在传统机器人研究领域, 实现足类机器人尤其是双足类机器人的行走是一个极具挑战的问题。和只有

六个关节的工业机器人操作臂相比, 人形机器人一般拥有几十个关节。之前应用于操作臂上的传统控制算法或规划方法不能直接应用于人形机器人, 一般要对人形机器人进行简化建模, 然后基于简化模型进行控制, 是一项相当复杂的工作。

基于此, 基于强化学习的方法学习走路对于人形机器人而言就意义重大。如图 6-5 为双足机器人使用强化学习算法学习走路。

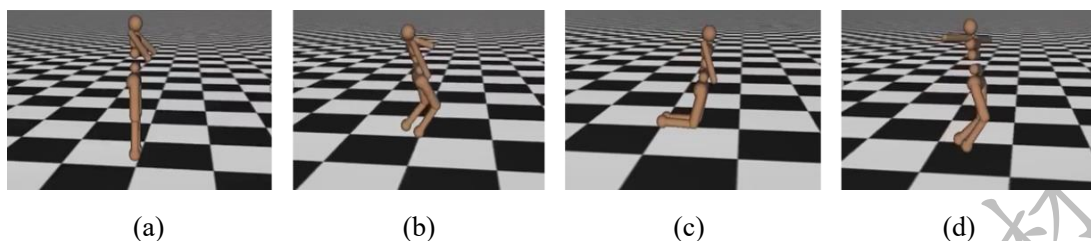


图 6-5 双足机器人学习走路的过程

与传统基于模型的方法不同, 基于强化学习的算法在设计控制器时不需要显式地对机器人进行困难的动力学建模, 换言之控制器的设计不需要机器人复杂的动力学模型。但是在仿真学习过程中, 需要一个可以对机器人运动进行仿真模拟的物理引擎。该物理引擎一般采用多刚体动力学模型, 考虑了机器人与地面之间的摩擦。在学习过程中, 首先初始化一个随机策略, 然后使用该随机策略与物理引擎进行交互, 将每一步的状态及其立即回报存储下来。机器人通过这种方式采集到大量的数据, 然后基于这些数据和强化学习的策略优化方法对当前的策略进行优化, 进而得到一个更好的策略。重复该过程, 使得策略不断得到优化。

仔细观察学习过程可以发现, 双足机器人在仿真系统中刚开始使用未训练的策略网络, 并不会走路, 无法保持平衡, 经常摔倒。摔倒后, 机器人得到负的回报, 利用这个负的回报机器人优化自身行走策略, 通过迭代优化, 策略网络变得越来越有效, 渐渐地机器人摔倒的次数越来越少, 往前运动的速度越来越快, 机器人逐渐学会了走路。

5. 核聚变控制^[6]

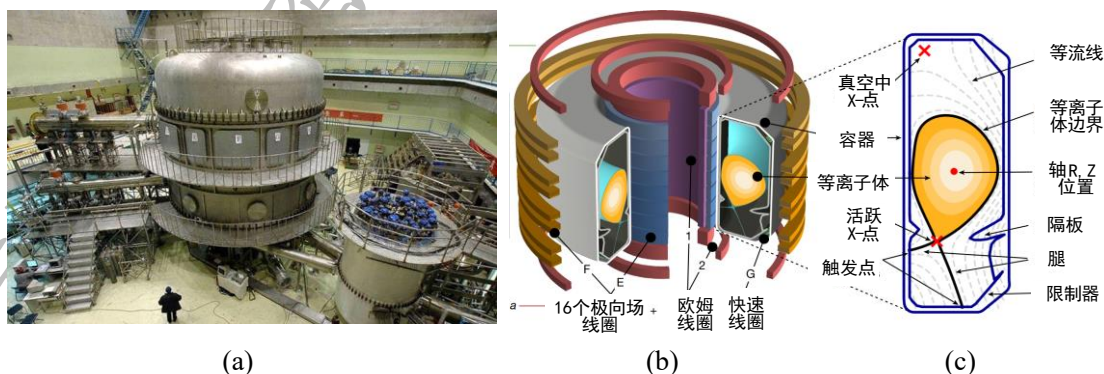


图 6-6 核聚变控制, a 为托马克装置, b 为托马克装置剖面, c 为等离子体截面

可控核聚变技术是人类实现清洁能源的重要核心技术之一。目前主要构型如图 6-6 所示, 其工作原理是采用超强电磁场将高速等离子体束缚在托马克装置内进行核聚变反应。

核聚变反应所释放出来的热量多少与正在受控反应的等离子体的截面形状(如水滴形、雪花形等)有关, 可控核聚变的技术关键就是调节电磁场来控制等离子体的截面形状。在本例中, 智能体需要根据当前系

统的状态，控制 19 个线圈的电压大小，从而调整电磁场进而改变等离子体的截面以满足放热需求。可控核聚变技术的基本思路是首先构建核聚变的模拟器，其次使用 Actor-Critic 的强化学习训练框架训练策略网络，最后将训练好的策略网络迁移到实际的托马克装置上控制 19 个线圈。

与其他任务相比，核聚变的线圈控制频率要求很高，一般要达到 10k 赫兹，即每个控制周期要小于 100 微秒，这给策略网络的计算带来巨大挑战。为了满足控制的实时性要求，策略网络参数不能太大，目前策略网络一般只用四层的前向神经网络。而 Critic 网络只在训练中使用，在部署和执行时不需要，因此 Critic 网络可以使用更大的循环神经网络。

6. ChatGPT^[7]

ChatGPT 是 OpenAI 公司于 2022 年 11 月发布的生成式预训练大规模语言模型，其功能包括聊天、撰写文章、编写代码等。ChatGPT 以 GPT 为基础，其中 GPT 全称为 Generative Pre-Trained Transformer。

“Generative”指的是生成式的，意味着这种模型能够生成文本；“Pre-trained”表示模型在大量数据上预先训练过，以学习语言的通用模式和结构；而“Transformer”是一种深度学习模型架构，它特别适用于处理序列数据，如文本。GPT 模型通过这种架构学习到的知识，能够生成连贯、相关且通常看起来十分自然的文本回复。

关于 ChatGPT 的训练，特别是 ChatGPT3.5 的版本，其训练过程可以分为如图 6-7 所示三个主要步骤：

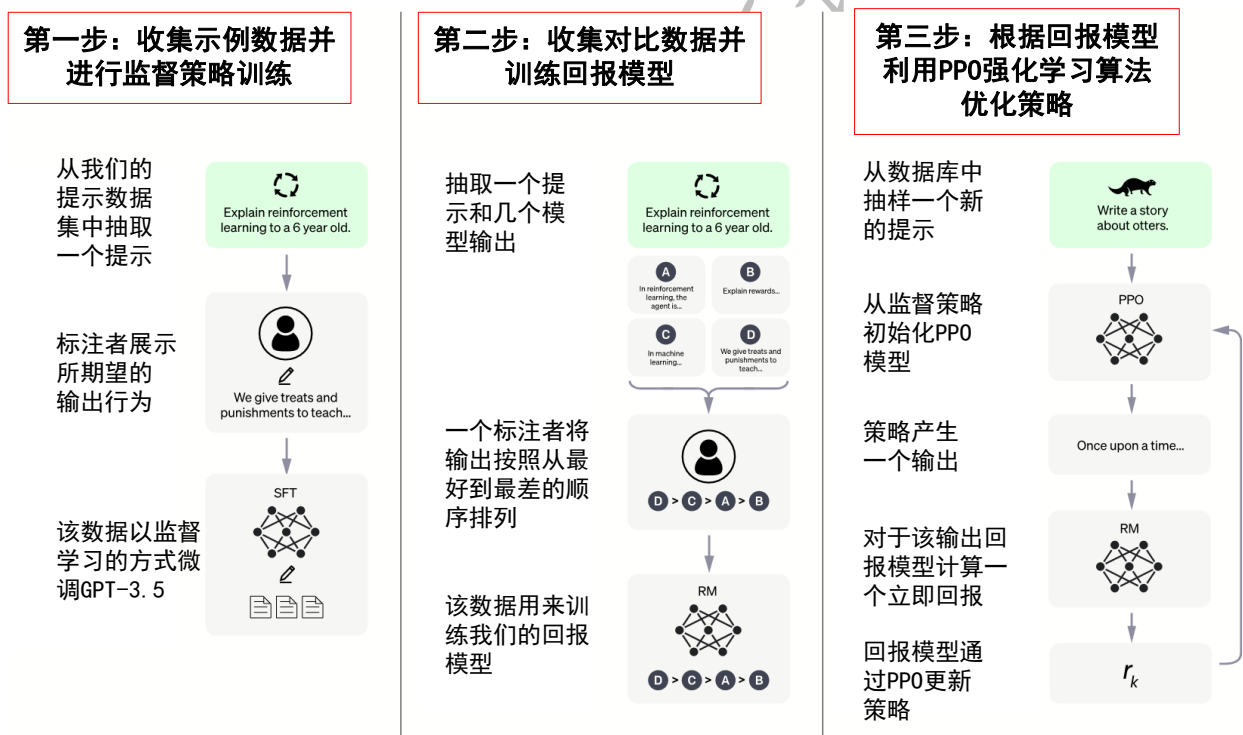


图 6-7 ChatGPT3.5 训练过程

第 1 步：有监督微调（Supervised Fine-Tuning, SFT）的数据收集

收集了大量的“提示（Prompt）和答复（Response）”数据对，部分来源于 OpenAI 提供的 Playground 平台上的互动，其余由专业的数据标注团队提供。此阶段的训练方法与 GPT-3 类似，旨在使模型能生成更自然和精确的文本回应。

第 2 步：人工标注对比数据集的构建与奖励函数模型的训练

模型首先生成一批候选文本，人工标注者根据这些文本的质量进行排序。对于每个提示（Prompt），ChatGPT 随机生成 K 个输出（ $4 \leq K \leq 9$ ），然后这些潜在回应将随机成对展示给数据标注专家，标注专家评估并排序这些回应，选择出他们认为更优的那一个。

第 3 步：利用 PPO 算法和学到的回报模型优化 SFT

使用一种强化学习算法 PPO（Proximal Policy Optimization，近端策略优化），它通过控制新旧策略比例的变化范围来优化模型。该算法通过限制新旧策略之比的下界对策略进行优化，在这个阶段，使用学到的回报模型作为环境反馈，通过 PPO 算法进一步优化和调整 SFT，以生成更高质量的文本输出。这一强化学习策略已在多种任务（例如机器人控制和多人在线游戏 DOTA 2）中证明了其有效性。

7. 星际争霸大师^[8]

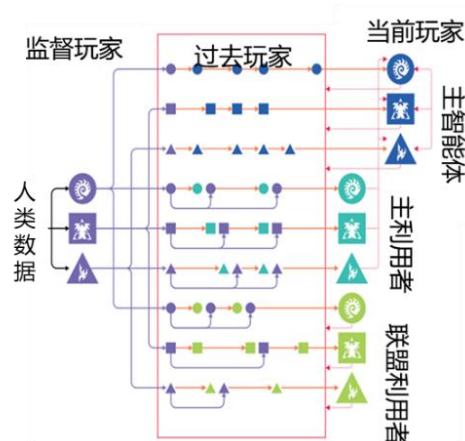
在前六个示例中，我们探讨了单一智能体系统应用强化学习的情景。然而，强化学习的技术也同样适用于多智能体系统的学习场景。相对于单智能体系统，多智能体系统展现出几个显著的差异点：

- 多智能体系统中的状态转移取决于所有智能体的行动。只有当每个智能体都作出决定后，系统状态才会发生变化。
- 对任一智能体而言，其他智能体被视作环境的一部分。然而，其他智能体本身也在不断地学习和调整策略，这使得整个系统对于任何单一智能体来说都是动态变化的，非稳态的。因此，多智能体环境对单一智能体来说并不符合马尔可夫性质。
- 在立即回报的角度，单智能体系统中智能体的立即回报仅受到自身行为的影响。相比之下，在多智能体系统中，每个智能体的立即回报不仅由自己的行为决定，还受到其他智能体行为的影响。

鉴于单智能体系统与多智能体系统之间的这些本质区别，通常无法直接将单智能体设计的强化学习算法应用于多智能体系统而期待同样的效果。



(a)



(b)

图 6-8 星际争霸大师 AlphaStar，(a) 星际争霸 2 对战画面，(b) 联盟训练

《星际争霸 II》是由暴雪娱乐开发的即时战略游戏，图 6-8 展示了针对《星际争霸 II》的 AlphaStar——一种具有划时代意义的 AI。在这个游戏中，玩家通过策略性地采矿、建设基地、组建军队并升级技术来与对手较量，目标是摧毁敌方的基地。比赛通常设置为 1 对 1 模式，玩家可以选择人族、神族或虫族进行对战。考虑到游戏的大规模和长期规划需求、信息的不完全性以及零和博弈特性，策略的复杂性使得为游

戏找到最优策略极具挑战。AlphaStar 通过训练，击败了 99.8% 的专业玩家，达到了大师级水平。

与单一智能体的训练相异，AlphaStar 使用了一种多智能体强化学习算法进行联盟训练，其联盟智能体由主智能体、主利用者和联盟利用者三部分组成，各自的职责如下：

- 主智能体：作为训练的中心，主智能体在训练中扮演多种角色，其中 35% 的时间用于自对抗，50% 的时间与以往的策略进行优先虚拟自对抗，剩余 15% 则与联盟利用者或之前的策略进行对抗。
- 主利用者：专注于挖掘主智能体的潜在弱点，进行训练时，与主智能体进行频繁的对战。
- 联盟利用者：旨在发现并利用整个联盟体系可能的漏洞，在训练中通过优先虚拟自对抗与联盟中的其他智能体交锋。

通过这种联盟训练，三个智能体同时进化，互相竞争，有效地解决了多智能体系统在学习过程中可能遇到的过拟合和策略循环问题。

表 6.1 强化学习的应用实例

序号	应用领域	实例描述	特点/策略
1	非线性控制系统	多级倒立摆的控制问题，如二级倒立摆的控制。	强化学习算法用于直接学习控制策略，无需复杂的动力学模型
2	视频游戏	如雅达利的打砖块游戏，DeepMind 的 DQN 算法被用来控制球拍。	结合强化学习和深度学习，超越人类玩家
3	围棋大师	DeepMind 的 AlphaGo 通过强化学习打败世界冠军。	使用策略网络和价值网络，以自对抗的方式进行学习
4	机器人学习走路	双足机器人使用强化学习算法学习走路。	通过交互学习控制策略，无需复杂动力学建模
5	核聚变控制	控制托马克装置中的等离子体。	实时性要求高，利用 Actor-Critic 框架
6	ChatGPT	OpenAI 的语言处理模型，结合监督学习和强化学习。	优化生成文本的质量
7	星际争霸大师	多智能体系统中，如《星际争霸 II》中的 AlphaStar。	多智能体强化学习算法，解决复杂策略问题

表 6.1 总结了强化学习在不同领域中的应用实例，展示了强化学习如何在各种序贯决策问题中找到最优策略。

6.1.2 强化学习能解决的问题

前面小节列举了目前强化学习技术可以解决的实际问题：为了让两级倒立摆的末端到达指定位置控制，智能体需要做的决策是台车在不同的系统状态时的运动受力方向；针对打砖块游戏，智能体需要做的决策是根据小球当前的运动状态和砖块的信息控制球拍的左右运动；为了赢得围棋比赛，智能体需要做的决策是如何在每一步落子；至于机器人学习走路，智能体需要做的决策是每个时刻如何控制关节电机以便机器

人稳定前向走动；在核聚变控制中，智能体需要做的决策是如何根据当前系统状态控制 19 个线圈的电压；ChatGPT3.5 中，智能体需要做的决策是如何根据对方的提问生成下一个词；星际争霸比较复杂，智能体需要做的决策是如何做出战略及战术动作以击败对手。分析上述实例可以发现，这些实例中需要解决的都是序贯决策最优问题。

所谓序贯决策最优问题（Sequential Decision Making）是指在连续的决策过程中，如何选择一系列行动使得某个性能指标（如累计奖励）最大化。在这个过程中，每个决策都可能依赖于前一个决策的结果，并会影响后续的决策和结果。

序贯决策最优涉及以下关键要素：

- **状态（State）**：在任何决策点，环境可以用一组状态来描述，状态包含了做出决策所需的所有相关信息。
- **行动（Action）**：智能体可以在每个状态下选择采取的行动。
- **策略（Policy）**：策略定义了在每个状态下应该采取的行动。最优策略是指导智能体在每个状态下选择能最大化长期奖励的行动。
- **奖励（Reward）**：智能体在转移到新状态后会收到一个立即奖励。总体目标通常是最大化长期奖励，即所有立即奖励的累计值。

序贯决策最优的问题可以用数学框架来表述，比如马尔可夫决策过程（MDP）。在 MDP 中，最优性通常定义为最大化预期累积奖励，也称为回报（Return），它是从当前时刻到未来某一时间点或无穷时间点的所有奖励的总和。当环境和智能体之间的交互遵循马尔可夫性质时，传统上可以用动态规划方法来求解最优策略。

从上节列举的 7 个应用实例可以看出，强化学习也比较擅长解决“序贯决策最优问题”。除了这 7 个应用实例，强化学习算法还可以应用到其他序贯决策领域，例如经济学中的投资决策、工业控制系统中的过程优化、游戏中的 AI 行为策略以及自动驾驶、视觉跟踪、机器翻译等智能决策领域。

6.2 强化学习与其他方法的区别和联系

本节重点介绍强化学习与监督学习的区别与联系，并介绍了强化学习算法与其他可以解决序贯决策问题的优化算法的联系与区别。

6.2.1 强化学习与监督学习的区别

监督学习是应用最广泛的类型，它的应用包括人脸识别、语音识别和手写数字识别等，这些都是日常生活中无处不在的技术。

监督学习的核心是利用标记的数据集来构建模型，并通过这些数据进行模型训练。这些数据集由特征输入和对应的标签组成，如图 6-9 所示，一张标记为数字“4”的手写数字图片用作输入和标签，标签在这里充当指导模型学习的监督信号。

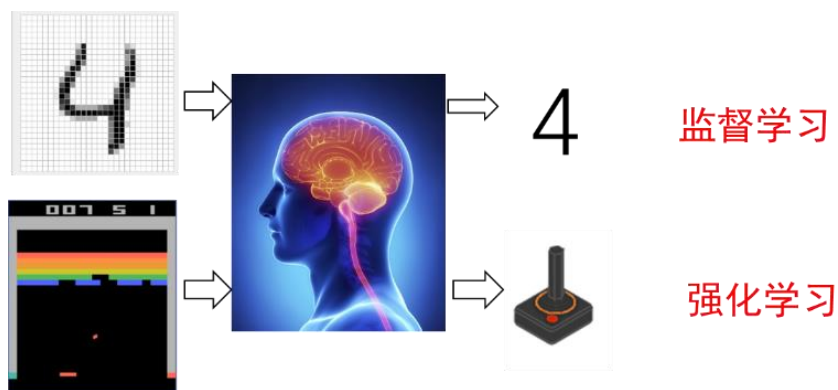


图 6-9 强化学习与监督学习的区别

与监督学习不同，强化学习在模型训练时并不需要知道当前状态下的最优动作标签是多少。强化学习使用的是由状态和回报组成的数据对。它依靠回报来评价一个动作的好坏，如果某个动作获得的回报高，则增加未来采取此动作的概率；反之，如果回报低，则减少。因此，从这个角度来看，回报是强化学习模型的一种弱监督信号。

强化学习和监督学习在目标、数据、反馈和环境等方面存在几个关键的区别：

1) 任务目标差异

监督学习主要处理瞬时感知任务，例如图像和语音识别。旨在学习输入数据到标签的映射关系。

强化学习解决的是长期（序贯）决策问题，如游戏玩法、自动驾驶等序贯决策任务。旨在学习一系列的行动，以最大化累计的奖励。

2) 数据类型差异

监督学习使用的是标记好的数据集，每个输入数据点都有一个正确的输出标签。

强化学习的数据来自于智能体与环境的互动，包含状态、行动和奖励。

3) 反馈机制差异

监督学习的反馈是直接的、明确的标签。

强化学习的反馈是奖励（或惩罚），这个奖励可能是延迟的，并且不总是直接指出最佳行动。

4) 学习环境差异

监督学习将数据集分为训练集和测试集，监督学习的环境是静态的，模型训练时不会改变数据分布。

强化学习在实施前需要一个环境模拟器，以生成序列数据，并用这些数据中的回报来迭代优化策略。强化学习的环境是动态的，在同一个环境中进行训练和测试，因为智能体需要在相同的环境中探索以发现最优的策略，同时也在这个环境中评估策略的效果。

简而言之，监督学习关注于准确预测给定数据的标签，而强化学习是通过智能体与环境的交互来学习一系列决策的过程，旨在长期获得最大奖励。

6.2.2 强化学习与监督学习的联系

强化学习和监督学习也存在着紧密的联系，具体表现如下：

1) 两者都使用数据从零开始学习

监督学习和强化学习在学习之前，其模型的参数都是随机给定的。通过采集数据和构建训练算法，两

者的模型性能不断提升，最终模型的智能水平可能超越人类。

2) 监督学习任务可以作为强化学习的辅助任务

比如在强化学习训练过程中，为了鼓励策略具有更强的探索性，往往将熵作为优化的目标放入到损失函数中。在基于模型的强化学习算法中，往往使用监督学习的方法进行模型的拟合。

3) 两者都使用梯度下降的方法优化模型

监督学习使用标签来构建损失函数，强化学习使用回报函数构建损失函数，最后进行模型更新时，使用的都是梯度下降的方法。

6.2.3 强化学习与其他优化方法的区别

如前所述，强化学习解决的是序贯决策问题。但强化学习并不是解决序贯决策问题的唯一优化方法。当状态空间很小，且动作离散，序贯决策问题可以建模为线性规划问题，这时候可以使用凸优化的方法。

另外，当状态转移模型已知，回报函数解析时，可以使用动态规划的方法来解决。如果状态转移方程为微分方程，只需要求解哈密尔顿-雅克比-贝尔曼方程即可。除此之外，我们可以将需要优化的策略进行参数化，然后使用遗传算法直接对策略进行优化。

然而，以上方法一方面受状态空间大小的限制，另一方面受状态转移模型及回报模型已知的限制。对于状态空间巨大、环境模型和回报模型未知的情况，我们只能使用强化学习算法。

跟其它方法不同，强化学习算法是基于采样的学习算法，它只需要环境样本就可以进行自身策略的优化。不需要使用显式的状态转移模型和回报函数模型进行策略的优化。这使得强化学习可以广泛地应用于各行各业。

遗传算法也经常用于策略的优化，强化学习与遗传算法的区别是遗传算法使用种群在整个状态空间的总体表现进行策略的优化；而强化学习利用策略在每个状态点处的评估函数来优化策略。换句话说讲，遗传算法关心整体，而强化学习算法更关心细节。

表 6.2 概括了强化学习与其他优化方法在解决序贯决策问题时的主要区别，包括各自的适用问题范围、对数学模型的需求、方法类型、学习方式和应用场景。

表 6.2 强化学习与其他优化方法在解决序贯决策问题时的主要区别

特性	强化学习	其他优化方法
问题范围	适用于大规模、复杂且模型未知的序贯决策问题	通常适用于状态空间小、模型已知的问题
数学模型需求	不需要明确的状态转移和回报函数模型	需要已知的状态转移和回报函数
方法类型	基于采样的学习方法，通过与环境交互采样来优化策略	线性规划、凸优化、动态规划等，依赖于明确的数学模型
学习方式	侧重于每个状态点的策略效用评估和优化	（如遗传算法）侧重于种群在整个状态空间中的整体表现

应用场景	适用于环境模型复杂或未知的情况，如自动驾驶、游戏等	适用于问题和环境模型明确、相对简单的情况，如某些工程和管理问题
------	---------------------------	---------------------------------

6.3 强化学习的思想精髓

强化学习是基于采样的方法进行学习。在介绍强化学习的基本学习思想之前，先介绍几个常用的概念。

策略（policy）^[9]：所谓策略是指给定任意状态 s ，指定该状态处动作集上的概率分布，即策略是状态空间向动作空间上的映射。

奖励(reward)：奖励是指在状态 s 处，做出动作 a ，环境给出的立即回报的期望。奖励一般是状态动作空间到实数空间的映射。

值函数（value function）：值函数 $V_{\pi}(s)$ 是指从该状态 s 开始，并采用策略 π 所得到的折扣累积回报的期望。

强化学习的目标就是找到使得值函数 $V_{\pi}(s)$ 最大的策略 π^* 。具体的采样过程如图 6-10 所示。首先智能体利用当前策略 $\pi(s)$ 采样动作 a ，然后将动作 a 作用到环境，环境则根据状态转移模型和回报模型给出新的状态和立即回报，重复该过程便能得到与环境交互产生的数据流：

$$s_0 \xrightarrow{a_0} (r_1, s_1) \xrightarrow{a_1} (r_2, s_2) \rightarrow \dots$$

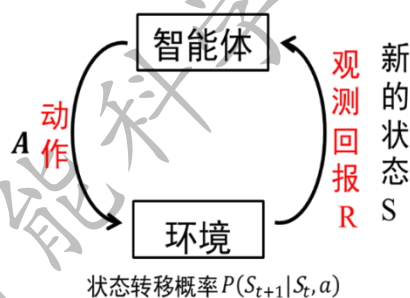


图 6-10 强化学习采样过程

智能体利用上述数据流进行学习。需要注意的是，采样的策略应该是具有随机性，这样才能尝试和探索其他的动作，进而找到最优策略。也就是说智能体的采样策略必须平衡探索（exploration）和利用（exploitation），这是强化学习能学到最优策略的精髓。

为了说明这个概念，我们再举一个井字游戏的例子。

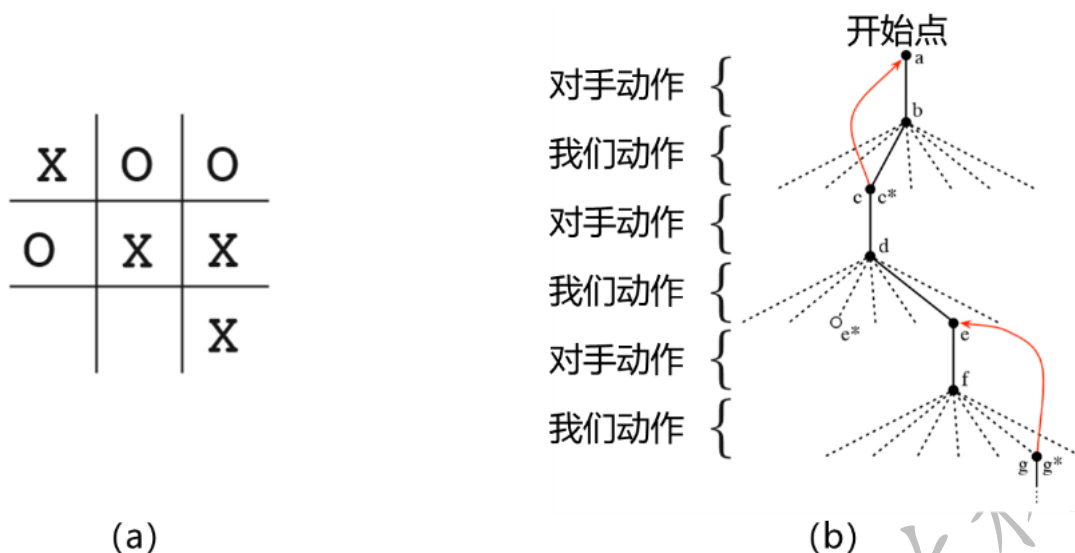


图 6-11 井字游戏采样过程

井字游戏如图 6-11 (a) 所示, 玩家在 3*3 的网格上落子, 胜负规则为优先在横向、纵向或斜向连成直线的一方胜利。如图 6-10 (b) 为智能体训练过程中的采样数据, 该对局的数据可表示为 $a \rightarrow b \rightarrow c^* \rightarrow d \rightarrow e \rightarrow f \rightarrow g^*$, 其中我方的决策点分别在 b, d 和 f, 决策点 b 处使用了“利用”动作, 即利用智能体当前的知识和经验, 采取了认为当前最好的动作 c^* 。在决策点 d 处, 智能体认为当前最好的动作应该是 e^* , 但是智能体并没有选择采用 e^* , 而是选择了使用非最优动作 e , 这时我们称智能体在此训练节点时采用了探索动作。探索动作对于找到最后策略是至关重要的, 因为智能体以当前的经验和知识所认为的最好的动作 e^* 并不一定是真实的最优动作。采用探索动作我们称为试错探索。智能体通过“探索-利用”平衡策略采样动作, 并通过该动作与环境进行交互, 得到环境返回的新状态及立即回报。

在强化学习算法中, 除了“探索-利用”平衡策略这个特点之外。强化学习任务往往还存在回报延迟的特点。如图 6-11 的井字游戏, 智能体在决策点 b、d 处做完决策后并没有得到游戏的回报, 只有当下棋结束后才知道胜负, 也才能对 b 和 d 处决策的动作进行评估。因此回报延迟以及如何使用折扣累积回报来评估动作是强化学习算法的另外一个精髓。

6.4 强化学习的发展方向

6.4.1 强化学习算法的发展回顾

我们不去深究强化学习算法的具体发展历史, 只给出两个关键的时间点。

第一个关键点是 1998 年, 标志性的事件是 Richard S. Sutton 出版了他的强化学习导论第一版, 即《Reinforcement Learning: An Introduction》^[10], 该书系统地总结了 1998 年以前强化学习算法的各种进展。在这一时期强化学习的基本理论框架已经形成。1998 年之前, 学者们关注和发展的最多的算法是表格型强化学习算法和基于值函数的强化学习算法。当然, 这一时期基于直接策略搜索的方法也被提出来了。如 1992 年 R.J.Williams 提出了 REINFORCE 算法直接对策略梯度进行估计^[11]。从 1998 年到 2013 年, 学者们发展出了各种直接策略搜索的方法。比如基于策略梯度的强化学习算法、基于 EM 的强化学习算法、基于路径

积分的强化学习算法等。

第二个关键点是 2013 年，Deepmind 提出 DQN，将深度网络与强化学习算法结合形成深度强化学习。2013 年后，随着深度学习技术的进步，各种深度神经网络被应用到强化学习算法中，比如使用深度神经网络表示策略网络或者表示值网络。之后，深度神经网络强大的表示能力加上强化学习强大的决策能力开启了具有超越人类决策能力的智能体。最典型的代表包括雅达利游戏水平达到人类玩家水平的 DQN，打败围棋世界冠军的 AlphaGo 系列，在《星际争霸》游戏中达到钻石级水平的 AlphaStar，具有颠覆性的 ChatGPT3.5 等。

6.4.2 现有强化学习算法的分类

已有的强化学习算法种类繁多，分类标准多样。表 6.3 通过对于强化学习算法的分类，展示了不同的强化学习算法类型及其特点。

表 6.3 强化学习算法的分类

分类标准	算法类型	特点描述
基于模型与无模型	基于模型的强化学习	学习环境的模型以预测未来状态和奖励，利用模型进行规划和决策。基于模型的强化学习算法效率要比无模型的强化学习算法效率更高。
	无模型的强化学习	直接从与环境的交互中学习如何行动，不构建环境的显式模型。无模型的强化学习算法因为不需要建模更具通用性。
基于策略更新和学习方法	基于值函数的强化学习	学习一个值函数来评估每个状态或状态-动作对，如 Q-learning，通过值函数来确定最佳策略。
	基于直接策略搜索的强化学习	直接优化策略本身，通常通过策略梯度方法实现，如 REINFORCE 算法。
	Actor-Critic 方法	结合值函数和直接策略搜索的方法，其中 Actor 根据当前策略执行行动，Critic 评估行动并指导 Actor 的学习。
回报函数是否已知	正向强化学习	回报函数是预定义的，智能体的目标是最大化累计奖励。
	逆向强化学习	回报函数未知，如无人机的特效表演，需要从专家行为或其他信息中自己推断回报函数。
其他强化学习方法	分层强化学习	通过构建多层决策框架来解决问题，每一层处理不同的决策粒度。

	元强化学习	关注如何快速适应新环境或任务的学习过程，强调学习如何学习的能力。
	多智能体强化学习	在多个智能体同时学习和互动的环境中，关注如何在竞争或合作的情境下最大化个体或群体的奖励。
	关系强化学习	利用关系和图结构的表示来处理状态，使智能体能在复杂环境中进行更有效的学习和决策。
	迁移强化学习	利用在一任务中获得的知识来加速在其他相关任务中的学习过程。

6.4.3 强化学习的发展趋势

近年来强化学习及相关技术已成为国际学术研究热点，其发展趋势值得注意：

1. 与深度学习的结合更加紧密

技术融合：2013 年深度强化学习（DRL）的兴起标志着深度学习与强化学习结合的加深，其中深度神经网络用于学习复杂的特征表示和决策策略。

应用突破：DRL 在多个领域实现了重大突破，如 2016 年采用了上百层残差网络的 AlphaGo 在围棋中的成功，2019 年采用记忆网络、指针网络等复杂网络体系的 AlphaStar 在即时战略游戏中的突破，以及 2022 年使用大语言模型的 ChatGPT 在自然语言处理中的突破性进展。

2. 与专业知识融合更加紧密

知识整合：强化学习与领域专业知识结合，如 AlphaGo 结合了深度学习与蒙特卡洛树搜索技术，显示出混合方法的强大潜力。

解决方案优化：结合专业领域的经验和方法可以提高强化学习算法的效率和有效性，更好地解决具体的应用问题。因此，如何与专业知识进行很好地融合是解决行业问题的发展趋势。

3. 与脑科学和认知神经科学的联系更加紧密

理论启发：从脑科学和认知神经科学获取灵感，可以帮助设计更先进的学习算法，这些算法能更贴近人类或动物的学习和决策过程。

记忆与学习：研究大脑如何处理记忆和学习可以启发强化学习算法的设计，特别是在处理复杂决策和长期记忆方面。当前大模型以其独特的性能已经具有超强的记忆，如何利用大模型的记忆能力来加快智能决策的学习也成为当前国际学术研究的热点问题。

4. 理论分析会更强，算法会更稳定和高效

理论进展：相关基础理论研究有助于更深入理解强化学习的原理和限制，从而设计出更稳定高效的算法。

算法改进：随着更多传统数学和控制领域的专家加入强化学习研究，可以预见算法在稳定性、效率和理论可解释性方面的显著提升。

上述趋势表明, 强化学习作为一个快速发展的研究领域, 不仅在技术和应用上取得了显著进展, 而且在理论分析和跨学科合作方面也展现出巨大的潜力和动力。随着研究的深入, 强化学习有望在更多领域中发挥核心作用, 推动人工智能技术的进步。

6.5 常用的强化学习算法简介

本节将介绍最常用的强化学习算法, 包括基于值函数的 Q-learning 算法、结合值函数与深度学习的 DQN 算法、策略梯度算法 PG、深度确定性策略梯度算法 DDPG、近端策略优化算法 PPO。

6.5.1 Q-learning 算法

Q-learning 是一种典型的基于值函数的强化学习算法, 通过维护一个行为值函数 $Q(S,A)$, 评估在特定状态 S 下选择动作 A 所能获得的预期收益。其核心在于使用一个二维的 Q 表, 其中一个维度表示状态, 另一维度表示动作, 对应位置上的数值即为当前状态-动作对的价值估计。经过多回合探索, 算法不断地根据采样得到的数据对 Q 表进行迭代更新, 逐步逼近真实的 Q 值, 直至收敛, 从而帮助智能体在不同状态下选择最优动作。

接下来, 我们利用伪代码来详细介绍 Q-learning 算法:

Q-learning 算法伪代码

1. 初始化 Q 表, 为所有的状态-动作对分配初始值 (通常为 0)
2. 对每一回合 (episode) 进行迭代:
 3. 初始化状态 S
 4. 选择动作 A // 基于当前状态 S , 利用某动作选择策略, 例如 ϵ -贪婪策略
 5. 迭代直至终止状态:
 6. 执行动作 A , 观察立即回报 R 和新状态 S'
 7. 选择新状态 S' 中具有最大 Q 值的动作 A'
 8. 更新 Q 表中的值 $Q(S,A)$:
$$Q(S,A) = Q(S,A) + \alpha[R + \gamma \max_{A'} Q(S',A') - Q(S,A)]$$
 10. $S \leftarrow S'$ // 更新执行动作 A 后到达的新状态
 11. 选择动作 A // 基于当前状态 S , 利用某动作选择策略, 例如 ϵ -贪婪策略
 12. 若达到终止状态 (如到达目标、超时、失败等), 则迭代结束
 13. 输出最终的 Q 表

Q-learning 算法的两个关键部分是算法的采样策略和评估策略。采样策略 (4、11 行) 是指在回合中产生数据时所采取的策略, 其需恰当地平衡探索与利用, 以便在尝试不同动作的同时找到最优策略。常用的是 ϵ -贪婪策略, 有一定概率采取其他策略进行探索, 也有一定概率采取当前最优策略进行利用。

评估策略（9 行）是指要被更新的策略，它是算法的最终产物，使被训练后的模型具备选择最优策略的能力。在评估策略的更新过程中，学习率 α 和折扣因子 γ 均为关键超参数。被广泛认可的评估策略主要包括蒙特卡洛方法和时间差分方法（伪代码中给出的是后者）。蒙特卡洛方法通过完整轨迹的回报来评估策略，虽得到无偏估计但方差较大；时间差分方法则结合当前步骤的回报与后继状态的值函数估计，虽然估计有偏但方差相对较小。

此外，根据评估策略和采样策略是否一致，强化学习可分为同策略（on-policy）和异策略（off-policy）方法。例如，SARSA 算法属于同策略，采样和评估策略相同；而 Q-learning 属于异策略，允许采样和评估策略不同（伪代码中，采样策略使用的是 ϵ -贪婪策略，而评估策略使用的是贪婪策略）。

6.5.2 DQN 算法

在 Q-learning 算法基础上，深度 Q 网络（Deep Q-learning Network, DQN）算法的核心改进是引入深度神经网络来表示行为值函数。这一改善的主要原因是，当状态空间和动作空间较大时，Q-learning 算法的 Q 表会很大，浪费资源且难以收敛，而神经网络理论上可以通过有限参数来近似拟合无限大的空间，同时也可应用于状态空间连续的问题。

此外，DQN 算法还进行了其他两项细节改进。其一，是经验回放机制，此机制借鉴自人类睡眠中的记忆整合过程，通过将智能体的经历存储在数据库中并随机抽取这些记录来训练网络，进而打破数据样本之间的时间关联、满足机器学习中的独立同分布假设，从而优化训练结果，促使神经网络的训练收敛且稳定。其二，DQN 算法引入了一个滞后更新的目标值函数网络，这一设计减少了学习过程中的目标值波动。该网络的参数不是在每次学习更新时都变化，而是每隔一定步数更新一次，进一步稳定了训练过程。

接下来，我们利用伪代码来详细介绍基于时间差分的 DQN 算法：

DQN 算法伪代码

1. 使用随机权重 θ 初始化值函数网络 Q
2. 初始化目标值函数网络 $\theta^- = \theta$
3. 初始化经验回放池 D 为空
4. 对每一回合（episode）进行迭代：
 5. 初始化状态 S
 6. 选择动作 A // 基于当前状态 S ，利用某动作选择策略，例如 ϵ -贪婪策略
 7. 迭代直至终止状态：
 8. 执行动作 A ，观察立即回报 R 和新状态 S'
 9. 选择新状态 S' 中具有最大 Q 值的动作 A'
 10. 将 (S, A, R, S', A') 存储到经验回放池 D 中
 11. 从 D 中随机采取小批量数据 ϕ // 设共有 n 个数据
 12. 对每个数据 $\phi_i = (S_i, A_i, R_i, S'_i, A'_i) \in \phi$ ：

13. 用以 θ 为参数的值函数网络估计 $Q(S_i, A_i | \theta)$
14. 用以 θ^- 为参数的目标值函数网络计算 $y_i = R_i + \gamma \max_{A'} Q(S'_i, A' | \theta^-)$
15. 计算 ϕ_i 在 θ 上的梯度 $\nabla_{\theta}[(y_i - Q(S_i, A_i | \theta))^2]$
16. 通过梯度下降更新值函数网络的参数:

$$\theta = \theta - \alpha \frac{1}{n} \sum_i^n \nabla_{\theta}[(y_i - Q(S_i, A_i | \theta))^2]$$
17. 每隔 C 步, 滞后更新一次目标值函数网络的参数 $\theta^- = \theta$
18. $S \leftarrow S'$ // 更新执行动作 A 后到达的新状态
19. 选择动作 A // 基于当前状态 S , 利用某动作选择策略, 例如 ϵ -贪婪策略
20. 若达到终止状态 (如到达目标、超时、失败等), 则迭代结束
21. 输出以 θ 为参数的值函数网络 Q

DQN 算法与 Q-learning 算法本质相同, 都是在探索环境 (5 行-10 行) 的同时更新值函数 Q (11 行-16 行), DQN 的一个鲜明特点在于, 其没有仅利用当前轮次的单次探索仅更新 Q 表中的对应状态, 而是利用了之前轮次的批量探索数据对值函数网络进行梯度下降。

6.5.3 策略梯度算法

在强化学习中, 除了 Q-learning 和 DQN 这种基于值函数(value-based)的方法, 还有一支非常经典的方法, 那就是基于策略(policy-based)的方法。对比两者, 基于值函数的方法主要是学习值函数, 然后根据值函数导出一个策略, 学习过程中并不存在一个显式的策略; 而基于策略的方法则是直接显式地学习一个目标策略。策略梯度(Policy Gradient, PG)是一个典型的基于策略的算法。

基于 TD 的 PG 算法和基于 TD 的 DQN 算法的伪代码实际上近似相同, 仅在网络参数更新时 (分别对应上节 DQN 伪代码的 16 行与本节 PG 伪代码的 9 行) 所有差别。为了拓阔读者视野, 以下给出基于蒙特卡洛方法的策略梯度方法 REINFORCE(REward Increment = nonnegative Factor $\times$ Offset Reinforcement $\times$ Characteristic Eligibility)算法的伪代码, 其通过完整轨迹的采样来计算累计回报 (4-8 行), 利用数据更新参数网络 (9 行)。

PG 算法伪代码

1. 使用随机权重 θ 初始化可微的参数化策略网络 $\pi(A | S, \theta)$
2. 初始化经验回放池 D 为空
3. 对每一回合 (episode) 进行迭代:
4. 初始化状态 S_0
5. 从 $i = 0$ 迭代直至终止状态 (如到达目标、超时、失败等) T :

6. 利用 $\pi(\cdot | S_i, \theta)$ 选择动作 A_i , 观测立即回报 R_i
7. 执行动作 A_i , 到达新状态 S_{i+1}
8. 计算本回合中每步的累计回报 $G_i = \sum_{j=i}^T (\gamma^{j-i} R_j)$, 并分别保存到 D 中
9. 从经验回放池 D 随机采样 M 个数据以更新网络参数:

$$\theta = \theta + \alpha \frac{1}{M} \sum_{i=0}^M \nabla_{\theta} \log \pi(A_i | S_i, \theta) G_i$$
10. 输出以 θ 为参数的策略网络 $\pi(A | S, \theta)$

值得注意的是, 相对于状态空间和动作空间都必须为离散的 Q-learning 算法, DQN 算法允许状态空间连续, 而 PG 算法允许动作空间连续。此外, 笔者以为, PG 和 DQN 算法的一个鲜明区别是对收敛的定义不同: DQN 算法中梯度下降的目标是通过调整网络参数降低当前状态的估计值和计算值 (分别对应上节伪代码中的 13 和 14 行) 的差值, 当差值无显著变化时, 算法收敛; 而 PG 算法通过调整网络参数来改变动作的概率分布, 以寻找一个最优策略来最大化这个策略的期望回报, 当参数优化无法带来回报的显著增长时, 算法收敛。

6.5.4 深度确定性策略梯度算法

本书之前的章节讲解了基于值函数的方法 (DQN) 和基于策略的方法 (PG), 其中基于值函数的方法只学习一个价值函数, 而基于策略的方法只学习一个策略函数。那么, 一个很自然的问题是, 有没有什么方法既学习价值函数, 又学习策略函数呢? 答案就是 Actor-Critic 架构。其囊括了一系列算法, 目前很多高效的前沿算法都是基于 Actor-Critic 架构。

深度确定性策略梯度算法 (Deep Deterministic Policy Gradient, DDPG)^[12] 是 Actor-Critic 架构下一个经典的强化学习算法, 专为高维动作空间设计。从某种角度看, DDPG 算法可以看作 PG 与 DQN 的结合, Actor 行动策略与 PG 近似, Critic 评估策略与 DQN 的值函数网络近似, 这种架构允许 DDPG 被用于动作空间和状态空间均连续的任务。

接下来, 我们利用伪代码来详细介绍 DDPG 算法:

DDPG 算法伪代码

1. 使用随机权重 θ^Q 初始化 Critic 网络 $Q(S, A | \theta^Q)$ 、 θ^μ 初始化 Actor 网络 $\mu(S | \theta^\mu)$
2. 初始化目标 Critic 网络和目标 Actor 网络的参数 $\theta^{Q-} = \theta^Q$, $\theta^{\mu-} = \theta^\mu$
3. 初始化经验回放池 D 为空
4. 对每一回合 (episode) 进行迭代:
 5. 初始化状态 S
 6. 选择动作 A // 根据当前 Actor 和探索噪声 $\mu(S | \theta^\mu) + \mathcal{N}$
 7. 迭代直至终止状态:
 8. 执行动作 A , 观察立即回报 R 和新状态 S'

10. 将 (S, A, R, S') 存储到经验回放池 D 中
11. 从 D 中随机采取小批量数据 ϕ // 设共有 n 个数据
12. 对每个数据 $\phi_i = (S_i, A_i, R_i, S'_i) \in \phi$:
13. 用 θ^Q 为参数的值函数网络估计 $Q(S_i, A_i | \theta^Q)$
14. 用 θ^{Q^-} , θ^{μ^-} 为参数的目标值函数网络计算 $y_i = R_i + \gamma Q(S'_i, \mu(S' | \theta^{\mu^-}) | \theta^{Q^-})$
15. 通过最小化损失函数 $L = \frac{1}{n} \sum_i^n (y_i - Q(S_i, A_i | \theta^Q))^2$ 更新 Critic 参数 θ^Q
16. 采用确定性策略梯度更新 Actor 参数 θ^μ , 其梯度为:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta^\mu} \mu(S_i | \theta^\mu) \nabla_a Q(S_i, a | \theta^Q) |_{a=\mu(S_i | \theta^\mu)}$$
17. 每隔 C 步, 滞后更新一次目标 Critic 和 Actor 网络参数:

$$\theta^{Q^-} = \tau \theta^Q + (1 - \tau) \theta^{Q^-}$$

$$\theta^{\mu^-} = \tau \theta^\mu + (1 - \tau) \theta^{\mu^-}$$
18. $S \leftarrow S'$ // 更新执行动作 A 后到达的新状态
19. 选择动作 A // 根据当前 Actor 和探索噪声 $\mu(S | \theta^\mu) + \mathcal{N}$
20. 若达到终止状态 (如到达目标、超时、失败等), 则迭代结束
21. 输出以 θ^Q 为参数的 Critic $Q(S, A | \theta^Q)$ 和以 θ^μ 为参数的 Actor $\mu(S | \theta^\mu)$

其中, 5-10 行与环境交互来生成数据, 11-15 行更新 Critic 网络, 16 行利用时间差分来更新 Actor 网络, 这里的梯度计算用到了链式法则。DDPG 算法同样采用 DQN 的滞后更新 (2、17 行) 和经验回放机制 (3、10、11 行)。

6.5.5 近端策略优化算法

近端策略优化算法 (Proximal Policy Optimization, PPO)^[13] 是另一种广泛使用的算法, 旨在解决随机策略梯度算法更新步长难以确定的问题。PPO 吸收了其前序算法 TRPO (Trust Region Policy Optimization) 所提出的优势函数, 与 DQN 的最小二乘法、PG 的最大化期望不同, 优势函数的核心思想是立足于现有旧策略, 找到一个比旧策略有最大优势的新策略。其次, PPO 创造性地引入了一种新的替代回报函数, 并通过限制策略更新幅度, 以维持训练过程的稳定性。替代回报函数结合了截断技术、值函数损失以及熵贡献, 它的目标是最大化期望回报的下界, 确保在优化过程中, 新策略相对于旧策略的性能不会急剧下降, 从而提高策略更新的稳定性。

接下来, 我们利用伪代码来详细介绍 PPO 算法:

PPO 算法伪代码

1. 使用随机权重 θ^w 初始化值函数网络 $V(S | \theta^w)$ 、 θ^μ 初始化策略网络 $\mu(A | S, \theta^\mu)$

2. 初始化目标策略网络 $\theta^{\mu^-} = \theta^{\mu}$
3. 初始化经验回放池 D 为空
4. 对每一回合 (episode) 进行迭代:
 5. 从环境中采集样本存储到 D 中 // 与其他算法同理, 此处不再赘述
 6. 从 $k = 1$ 逐次递增至 K :
 7. 从 D 中随机采取一个批次的数据 (S, A, R, S')
 8. 计算优势函数 $\mathcal{A}(S, A)_k = R + \gamma V(S' | \theta^w) - V(S | \theta^w)$
 9. 计算动作概率比值 $ratio_k = \mu(A | S, \theta^{\mu}) / \mu(A | S, \theta^{\mu^-})$
 10. 根据如下优化方向, 更新 θ^{μ} :

$$\theta^{\mu} = \operatorname{argmax}_{\theta^{\mu}} \frac{1}{K} \sum_{k=1}^K [\min(ratio_k \mathcal{A}(S, A)_k, \operatorname{clip}(ratio_k, 1 - \varepsilon, 1 + \varepsilon) \mathcal{A}(S, A)_k)]$$
 // ε 为较小值, clip 为截断函数, 将 $ratio_k$ 截断到较小范围
11. 根据最小二乘的梯度, 更新 θ^w :

$$\theta^w = \theta^w - \alpha \frac{1}{K} \sum_{k=1}^K \nabla_{\theta} [(R_k + \gamma V(S'_i | \theta^w) - V(S_i | \theta^w))^2]$$
12. 每隔 C 步, 滞后更新一次目标策略网络 $\theta^{\mu^-} = \theta^{\mu}$
13. 输出最终以 θ^{μ} 为权重的策略网络 $\mu(A | S, \theta^{\mu})$

其中, 6-10 行计算策略网络的更新方向, 11 行更新值函数网络的更新方向。通过限制策略更新量 (10 行), PPO 算法不仅能够提高学习的稳定性, 还能有效应对环境中的不确定性和变化, 使算法在各种任务中表现出色。

上述这些算法在不同的强化学习场景和问题中有各自的优势和应用。选择合适的算法通常取决于特定问题的特性, 如状态空间的大小、是否需要模型、任务的序贯性等。表 6.4 总结了这几种典型的强化学习算法及其特点。

表 6.4 几种典型的强化学习算法及其特点

算法名称	类型	特点	应用场景
Q-learning	基于值函数的无模型	利用贪婪策略更新值函数, 适用于离散动作空间	经典强化学习任务, 如迷宫导航、简单的游戏
DQN (Deep Q-Network)	深度学习扩展的 Q-learning	结合深度神经网络和经验回放, 处理高维状态空间	视频游戏、机器人导航、复杂决策问题
策略梯度算法 PG	基于策略的方法	直接优化策略函数, 适用于连续动作空间	机器人控制、股票交易、高维动作空间的任務
深度确定性策略	Actor-Critic 方法	结合值函数和策略的优点, 适用	自动驾驶、机器人操控、

梯度算法 DDPG		于连续状态空间、连续动作空间	复杂模拟环境
近端策略优化算法 PPO	改进的策略梯度方法	使用截断的策略梯度和多步更新，平衡效率和稳定性	各种环境中的连续控制任务、游戏 AI

除了上述提到的 Q-learning、DQN、策略梯度算法 (PG)、深度确定性策略梯度算法 (DDPG) 和近端策略优化算法 (PPO)，还有一些其他算法也可以归入强化学习范畴，例如：

- 蒙特卡洛方法 (Monte Carlo Methods)：是强化学习中的一种基本方法。20 世纪 40 年代，科学家斯塔尼斯拉夫·乌拉姆 Stanislaw Ulam、冯·诺伊曼 John von Neumann 和尼古拉斯·梅特罗波利斯 Nicholas Metropolis 于美国洛斯阿拉莫斯国家实验室为核武器计划工作时，发明了蒙特卡罗方法。该方法是不基于模型的算法，通过对环境的采样来估计值函数。适用于那些可以明确定义完整情节的任务，通过完整的情节来更新价值估计。
- 时间差分学习 (Temporal difference learning, TD learning)：结合了动态规划和蒙特卡洛方法的优点，可以在不知道环境模型的情况下学习。典型的算法有 SARSA 和 TD(λ)。
- SARSA (State-Action-Reward-State-Action)：一种基于模型的 TD 学习方法，考虑了当前状态和行为后的下一个状态和行为。
- TRPO (Trusted Region Policy Optimization)：一种在策略空间中进行优化的算法，通过限制策略更新的步长来避免过大的性能波动。
- A3C (Asynchronous Advantage Actor-Critic)：使用多线程来收集不同环境的经验，加速学习过程。结合了 Actor-Critic 框架和深度学习，能够处理高维观测空间。
- Rainbow：结合了多种深度强化学习技术的综合算法，如双 DQN (Double DQN)、优先经验回放 (Prioritized Experience Replay)、多步学习 (Multi-step Learning) 等。
- Soft Actor-Critic (SAC)：基于最大熵强化学习框架，旨在同时最大化预期回报和策略的熵，促使策略更加探索性。
- MuZero：一种结合了模型预测和模型无关的强化学习优点的算法，能在没有环境模型的情况下学习如何预测环境的动态。

6.6 强化学习的术语与总结

本节给出强化学习的相关术语，并给出本章小结。

6.6.1 相关的术语

智能体 (agent)：在强化学习中，智能体是指与环境进行交互的实体，其目标是通过这些交互学习最优策略，以最大化其累积回报。智能体通过观察环境状态，根据所学策略做出决策（即选择动作），并接收环境提供的回馈（通常是奖励）。通过这种反馈，智能体不断调整其行为，以改进其决策过程和性能，适应环境并实现长期目标。这一过程涉及到决策模型的建立、策略的评估和优化，使得智能体能够在不断变化的环境中做出最佳决策。

策略 (Policy): 是强化学习中智能体在给定状态 S 下选择动作的规则。更具体地，策略 π 定义为一个从状态空间到动作空间的概率分布映射，即在每一个状态下，策略确定了智能体选择每个可能动作的概率。在确定性策略中，策略直接映射状态到具体动作，即在每个状态下始终采取同一个动作；在随机策略中，策略为每个状态分配一个概率分布，从而以不同的概率选择不同的动作。通常，用 $\pi(a | s)$ 表示策略在状态 s 下选择动作 a 的概率。策略的优化目标是使智能体获得最大累计奖励，从而学得最优策略。

同策略 (On-policy): 被评估的策略与用于采样的策略为同一个策略。

异策略 (Off-policy): 在异策略学习中，评估的策略与用于生成行为数据的策略不同。这允许从一个策略的行为中学习并优化另一个策略。

值函数 (Value Function): 是强化学习中用于估计智能体在某个状态 S 下的预期回报的函数。具体来说，值函数衡量了从当前状态 S 出发，按照特定策略 π 执行动作后，智能体能够期望获得的累积奖励。这种累积奖励通常被称为**回报 (return)**，并且随着时间步长进行折扣（如使用折扣因子 γ ）。有两种主要的值函数，状态值函数 $V^\pi(S)$ 和行为值函数 $Q^\pi(S, A)$ 。

马尔可夫性 (Markov property): 是指一个系统的未来状态仅依赖于其当前状态，而与它的过去状态或是如何达到当前状态无关。这是马尔可夫决策过程的核心属性。这种性质表明，系统的未来发展只与当前瞬间的情况有关，而与历史路径无关。

在数学术语中，如果一个过程具有马尔可夫性，那么可以用以下条件概率公式来表示：

$$P(S_{t+1} | S_t, S_{t-1}, \dots, S_0) = P(S_{t+1} | S_t)$$

这里， S_{t+1} 是下一时刻的状态， S_t 是当前时刻的状态，而 $S_{t-1}, \dots, S_0$ 表示过去的状态序列。这个公式说明了下一状态的概率只依赖于当前状态。

马尔可夫性是许多统计模型和随机过程分析中的基础假设，尤其是在马尔可夫链 (Markov chain) 中，这是一种特殊类型的随机过程，其中的每一状态都具有马尔可夫性质。在马尔可夫链中，过程的未来演变只取决于当前的状态，并不依赖于它是如何进入这个状态的。

马尔可夫决策过程 (MDP): 在强化学习中，马尔可夫决策过程是一种数学框架，用于形式化环境中的决策问题。有限马尔可夫决策过程由五元组 (S, A, P, R, γ) 描述， S 为状态集，表示可能的环境状态； A 为有限的动作集，表示可执行的动作； P 为状态转移概率，表示从一个状态转移到另一个状态的概率，状态之间的转移满足马尔可夫性； R 为立即回报函数，表示执行动作后接收到的立即回报； γ (Gamma) 为折扣因子，用于计算未来回报的当前价值，表明未来奖励的重要性。

6.6.2 总结

本章首先举例介绍了强化学习算法能解决的问题，明确了强化学习针对的是序贯决策问题。接着介绍了强化学习与其他方法的区别和联系，包括与监督学习的相同点和不同点，与其他可以解决序贯决策问题的优化方法的区别。然后阐述了强化学习思想的精髓是试错探索，并简单地介绍了强化学习发展历史及趋势。最后，介绍了几种常用的强化学习算法。强化学习技术在快速发展，尤其是跟大模型结合形成的决策大模型技术必将发挥越来越大的作用。

6.7 思考

【思考 6.1】 蒙特卡洛方法 (Monte Carlo Methods) 可以追溯到 20 世纪 40 年代, 为什么也属于强化学习中的一种基本方法?

【思考 6.2】 请解释策略 (policy) 与值函数 (value function) 之间的关系, 并描述在强化学习中如何使用值函数来改进策略。

【思考 6.3】 在强化学习中, 为什么需要平衡探索 (exploration) 和利用 (exploitation)? 请给出一个实际的例子来说明这两者的重要性。

【思考 6.4】 试着用伪代码或流程图描述 Q-learning 算法的步骤。在描述中, 请指出在哪一步会发生探索, 在哪一步会发生利用。

6.8 习题

【作业 6.1】 强化学习与认知神经科学的关系是什么?

【作业 6.2】 强化学习与深度学习如何共同推动机器学习的发展?

【作业 6.3】 选择一个实际领域 (如自动驾驶、金融交易等), 描述强化学习如何被用来解决该领域的具体问题, 并讨论可能面临的挑战。

参考文献

- [1] DEISENROTH M P, RASMUSSEN C E. PILCO: A Model-Based and Data-Efficient Approach to Policy Search: ICML 2011: proceedings of the 28th International Conference on Machine Learning, Bellevue, Washington, USA, June 28 - July 2, 2011[C]. Omnipress, c2011.
- [2] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518: 529–533 .
- [3] SILVER D, HUANG A, MADDISON C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 2016, 529:484-489.
- [4] SILVER D, SCHRITTWIESER J, SIMONYAN K, et al. Mastering the game of go without human knowledge[J]. Nature, 2017, 550: 354-359.
- [5] SCHRITTWIESER J, ANTONOGLOU I, HUBERT T, et al. Mastering Atari, Go, chess and shogi by planning with a learned model[J]. Nature, 2020, 588, 604–609.
- [6] DEGRAVE J, FELICI F, BUCHLI J, et al. Magnetic control of tokamak plasmas through deep reinforcement learning[J]. Nature, 2022, 602, 414–419 .
- [7] <https://openai.com/blog/chatgpt/>
- [8] VINYALS O, BABUSCHKIN I, CZARNECKI W M, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning[J]. Nature, 2019, 575, 350–354.
- [9] SUTTON R S, BARTO A G. Reinforcement Learning An Introduction(Second Edition)[M]. Cambridge, Massachusetts: The MIT Press, 2018.
- [10] SUTTON R S, BARTO A G. Reinforcement Learning An Introduction[M]. Cambridge, Massachusetts:

The MIT Press, 1998.

- [11] WILLIAM R. J. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning[J]. Machine Learning, 1992, 8:229–256 .
- [12] LILICRAP T P, HUNT J J, PRITZEL A, et al. (2016) Continuous control with deep reinforcement learning: ICLR 2016: proceedings of the International Conference on Learning Representation, Vancouver, Canada, May 2-4, 2016[C]. OpenReview. net, c2016.
- [13] SCHULMAN J , WOLSKI F , DHARIWAL P ,et al. Proximal Policy Optimization Algorithms[J]. Journal of Machine Learning Research, 2017,18(1) :1-48.

《自动化与智能科学概论》教材初稿